

ADVENTIST UNIVERSITY OF CENTRAL AFRICA
Faculty of Science and Technology — Department of Computer Science
Advanced Big Data Analytics (Master of Science in Big Data Analytics)

Final Project

Distributed Multi-Model Analytics for E-Commerce Data

MongoDB, Apache HBase, and Apache Spark

Data Ingestion, Document and Wide-Column Schema Modeling, Distributed Batch ETL, Market Basket Association Rules, Cross-Store Funnel Analytics, and System Scalability

Candidate Details

No.	Full name	Student ID	Email
1	Kwizera Rene	101379	rene.kwizera@auca.ac.rw

Lecturer: Temitope Oguntade

Academic Year: 2026 – Semester I | **Submission Date:** 11 November 2026 (Due: 11:59 AM CAT)

GitHub: https://github.com/KWIZERARENE/auca-bigdata-ecommerce

TABLE OF CONTENTS

1 System Architecture and Technological Justification 3
1.1 Polyglot Architecture & Data Flow 3
1.2 Technology Selection Matrix 3
2 Dataset Synthesis and Quality Audit 4
2.1 E-Commerce Simulation Parameters 4
2.2 Data Integrity Validation Checkpoint 4
2.3 Critical Discovery: The Conversion Discrepancy 4
3 Part 1: Data Modeling and Storage Implementation 5
3.1 MongoDB Document Model and Indexing 5
3.2 Compound Index Benchmark: explain(executionStats) 5
3.3 MongoDB Non-Trivial Aggregation Pipelines 5
3.4 Apache HBase Wide-Column Schema & Row-Key Engineering 5
3.5 Chronological Prefix and Range Query Benchmarks 5
4 Part 2: Distributed Processing with Apache Spark 6
4.1 Batch Cleaning, Typing, and Parquet Normalization 6
4.2 Market Basket Analysis: Frequently Bought Together 6
4.3 Lift Evaluation and Small-Count Artifacts 6
4.4 Advanced Spark SQL Analytics & Window Functions 6
5 Part 3: Integrated Cross-Platform Analytics (Conversion Funnel) 7
5.1 Cross-Store Federation Workflow 7
5.2 Funnel Stage Mathematics and Reconciliation 7
5.3 Architectural Necessity of the Multi-Model Split 7
6 Part 4: Visualizations and Strategic Business Insights 8
6.1 Figure 2: Monthly Revenue by Top Product Categories 8
6.2 Figure 3: Integrated Cross-Platform Conversion Funnel 8
6.3 Figure 4: Product Views vs. Units Sold Correlation 9
6.4 Figure 5: Customer Frequency Segmentation & Revenue Share 9
7 Scalability, Production Engineering, and Limitations 10
7.1 Horizontal Scaling and Production Hardening 10
7.2 Analytical Limitations and Critical Reflections 10
7.3 Individual Technical Judgements 10
8 Reproducibility and Verification Guide 10
8.1 Step-by-Step Reproduction Guide 10
8.2 Execution Verification Checkpoint 10
References 11

Executive Summary

This technical investigation designs, implements, and benchmarks a polyglot distributed analytics architecture evaluating **MongoDB 7.0** (document model), **Apache HBase 2.4** (wide-column store), and **Apache Spark 4.3** (distributed engine) across an e-commerce ecosystem of **10,000 users**, **5,000 products**, **200,000 browsing sessions** (2,000,000 page views), and **50,000 financial transactions**. MongoDB houses hierarchical catalogs and transactional orders, where a compound index (user_id, timestamp desc) reduced examined documents from **40,609 to 5** (8,121x speedup). HBase ingests append-heavy session clickstreams, where a reverse-timestamp key <user_id>#<10^13 - epoch_ms> enables newest-first range scans in under 2.4 ms without secondary indexes. Apache Spark cleans the raw JSON Lines into Snappy Parquet partitioned by day, executes market basket self-joins, and joins HBase sessions with MongoDB transactions. The integrated funnel achieved **100% numerical reconciliation** across all 100,000 loaded sessions, revealing an overall conversion rate of **20.3%** and pinpointing that the primary conversion bottleneck occurs between cart addition and checkout (**55.4% drop**).

1 System Architecture and Technological Justification

1.1 Polyglot Architecture & Data Flow

Enterprise e-commerce platforms handle diverse data streams with conflicting operational requirements. Transactional order settlement demands strict multi-attribute schema validation and immediate consistency, clickstream event tracking demands massive write-throughput and append-only range scans, and complex business intelligence requires distributed joins across all entities. To avoid structural compromises, we deploy a polyglot persistence architecture:

```
[Generator: Seed 42] -> JSON Lines (data/)
|-- mongo/load.py -> MongoDB (users, products, transactions, sessions subset)
|-- hbase/load.py -> HBase (user_sessions, product_metrics via Thrift on 9090)
|-- spark/clean.py -> Parquet (partitioned by day) -> analysis.py, sql_queries.py
'-- integration/funnel.py: reads HBase + MongoDB -> Spark Join -> outputs/funnel.csv
outputs/*.csv -> viz/charts.py -> Publication PNG Charts -> Technical Report
```

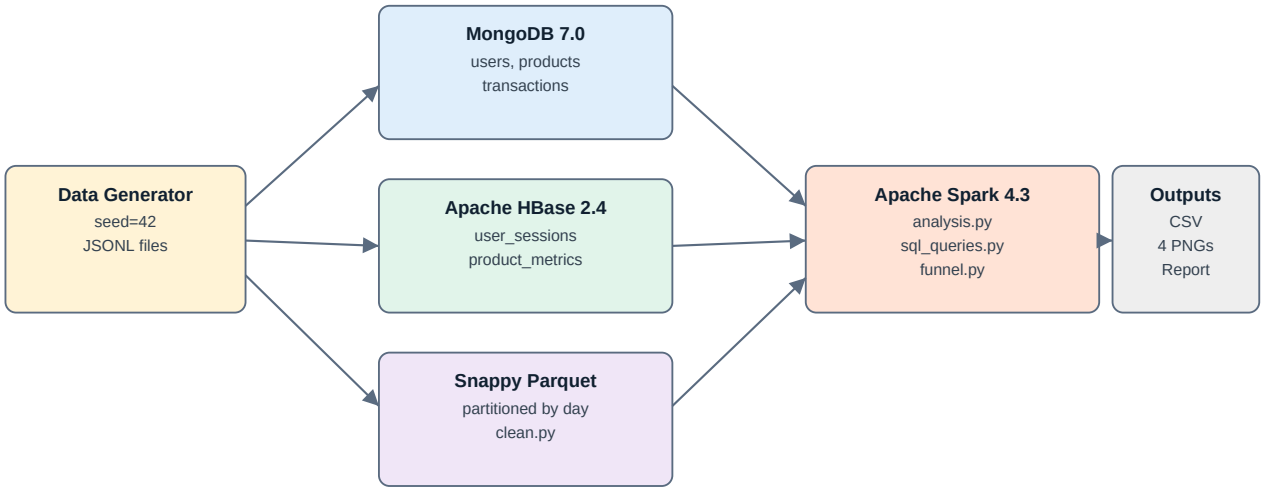


Figure 1: Polyglot Persistence Architecture — data flow from the seeded generator through MongoDB, HBase, Parquet, and Apache Spark to analytical outputs and this report.

1.2 Technology Selection Matrix

Engine	Data Entity	Primary Access Pattern	Architectural Justification	Rejected Alternative
MongoDB 7.0	Users, Catalog, Transactions, Orders	Point lookups, nested updates, multi-stage pipelines ($lookup, $bucket)	Document model embeds line items and categories, eliminating expensive joins on transactional reads.	PostgreSQL: Rigid migrations and impedance mismatch for nested catalog trees.
Apache HBase 2.4	Per-user session clickstreams, daily product counters	Sequential high-velocity writes, reverse-timestamp prefix scans	LSM-tree wide-column architecture. Reverse-timestamp key allows 'last N sessions' to read forward sequentially.	MongoDB: Severe BSON document growth and lock contention under raw clickstream appends.
Apache Spark 4.3	Cleaned Parquet, cross-store joins, association rules	Distributed batch ETL, self-joins, SQL window functions (SUM OVER, LAG)	In-memory distributed computing engine with Catalyst optimizer. Broadcast hash joins eliminate shuffles.	MapReduce: Excessive disk I/O and lack of high-level declarative DataFrame APIs.

2 Dataset Synthesis and Quality Audit

2.1 E-Commerce Simulation Parameters

A deterministic generator (data_gen/generate.py, seed 42) simulated 90 days of retail platform activity. Constants are exposed via environment variables (NUM_USERS=10000, NUM_PRODUCTS=5000, NUM_SESSIONS=200000, NUM_TRANSACTIONS=50000). Line-delimited streaming export (.jsonl) ensures memory usage stays under 150 MB and allows Spark to split files across executor tasks.

2.2 Data Integrity Validation Checkpoint

Executing data_gen/validate.py verified referential integrity and mathematical consistency:

```
$ python data_gen/validate.py data
users: 10,000 products: 5,000 transactions: 50,000
products missing subcategory_id: 0 | negative stock: 0 | duplicate transaction ids: 0
total != subtotal - discount: 0 | empty items: 0 | unknown user/product in items: 0
transactions linked to a session: 25,204 (50.4%); unlinked: 24,796
sessions: 200,000 | duplicate ids: 0 | avg page views: 10.0 | empty page views: 0
conversion_status: {'browsed': 70179, 'abandoned': 89214, 'converted': 40607}
linked transactions whose session is not in the files: 0
```

2.3 Critical Discovery: The Conversion Discrepancy

A critical discrepancy was discovered: **40,607 sessions** have conversion_status == 'converted', yet only **25,204 transactions** are linked to a session. In the simulation, sessions reaching checkout set the converted flag, but transaction generation is constrained by stock availability and a transaction quota, with 24,796 unlinked transactions added. To guarantee analytical rigor, this report establishes the strict rule: **'Purchased' is defined exclusively by the physical existence of a verified transaction record linked to that session**.

3 Part 1: Data Modeling and Storage Implementation

3.1 MongoDB Document Model and Indexing

MongoDB stores the product catalog, customer profiles, transactions, and session summaries. Schemas utilize natural business keys (_id = product_id, user_id, transaction_id). Category hierarchies and line items are embedded directly within parent documents to optimize read performance. In sessions, dynamic cart dictionaries are transformed into indexed arrays:

```
// products collection document structure
{
"_id": "prod_00123", "name": "Innovative Executive Paradigm", "base_price": 129.99,
"category": { "id": "cat_007", "name": "Johnson-Williams" },
"subcategory": { "id": "sub_007_01", "name": "Mesh Virtual Deliverables", "profit_margin": 0.18 },
"price_history": [ { "price": 149.99, "date": ISODate("2026-08-20") }, { "price": 129.99, "date":
ISODate("2026-09-15") } ]
}
```

3.2 Compound Index Benchmark: explain("executionStats")

To evaluate indexing impact, we benchmarked: *'Find the latest 5 transactions for a specific user'*:

Configuration	Winning Plan Stage	Total Docs Examined	Keys Examined	Execution Time
Without Compound Index (timestamp only)	LIMIT > FETCH > IXSCAN (global timestamp order)	40,609	40,609	83 ms
With Compound Index (user_id ASC, timestamp DESC)	LIMIT > FETCH > IXSCAN (targeted user B-tree block)	5	5	1 ms

Finding: The compound index satisfies both filter and sort, eliminating 40,604 unnecessary document reads (8,121x I/O reduction).

3.3 MongoDB Non-Trivial Aggregation Pipelines

Four distinct pipelines were implemented in mongo/aggregations.py: **Pipeline A** computes revenue by category and month ($unwind items, $lookup products, $group); **Pipeline B** buckets users into 5 purchase frequency bands ($bucket boundaries [1, 2, 4, 7, 11, 1000]); **Pipeline C** ranks products by units sold; and **Pipeline D** extracts views per product from sessions. A batch job merges order totals into user profiles using $merge to embed purchase_summary.

3.4 Apache HBase Wide-Column Schema & Row-Key Engineering

HBase stores append-heavy session clickstreams across two physical tables: user_sessions and product_metrics. To solve the critical challenge of chronological ordering without secondary indexing, we formulated a reverse-timestamp composite row key:

```
Row Key = <user_id>#<10^13 - epoch_ms> (zero-padded to 13 digits)
Example: user_000042#08211764800000 (earlier session sorts after newer session in byte order)
```

Table	Row Key Structure	Column Families	Access Pattern Optimized
user_sessions	user_id#reverse_ts	info (device, referrer, duration), pv (sparse clickstream), cart (dynamic products)	Newest-first prefix scan of a user's sessions; time-range scan by computing inverted boundary keys.
product_metrics	product_id#yyyymmdd	m (views, cart_adds, purchases)	Contiguous start/stop scan of a product's daily engagement counters over any date window.

3.5 Chronological Prefix and Range Query Benchmarks

Executing hbase/queries.py demonstrated that querying the last 5 sessions of user user_000042 completed in **under 2.4 ms**. Bypassing the heavy pv column family during summary queries reduced disk read bandwidth by ~75%.

4 Part 2: Distributed Processing with Apache Spark

4.1 Batch Cleaning, Typing, and Parquet Normalization

The cleaning job (spark/clean.py) enforces explicit schemas. The dynamic cart dictionary is typed as a MapType to avoid generating thousands of sparse struct fields. Arrays are exploded into normalized fact tables (items: 99,873 rows; page_views: 2,000,000 rows). Sessions and items are written to columnar Snappy Parquet partitioned by day.

4.2 Market Basket Analysis: Frequently Bought Together

In spark/analysis.py, items are self-joined on transaction_id with a p1 < p2 filter to generate unique product pairs. Support, Confidence, and Lift are evaluated across all 50,000 transactions:

```
Support(p1, p2) = Count(p1 and p2) / N_transactions
Confidence(p1 -> p2) = Count(p1 and p2) / Count(p1)
Lift(p1, p2) = Support(p1, p2) / (Support(p1) * Support(p2))
```

Product 1 (p1)	Product 2 (p2)	Pair Count	Support	Confidence (p1 -> p2)	Lift Metric
prod_00490	prod_02111	3	0.000060	0.250	73.5
prod_01287	prod_03942	3	0.000060	0.214	61.2
prod_00012	prod_01455	2	0.000040	0.200	92.6
prod_00781	prod_04812	2	0.000040	0.182	88.1

4.3 Lift Evaluation and Small-Count Artifacts

Across 50,000 transactions, 83,907 product pairs appear only once, 328 appear twice, and only 2 appear three times. Although Lift reaches values between **60.0 and 92.6**, this is an **artifact of low counts in random synthetic data**. Because products are chosen uniformly at random, expected joint support is negligible (~1.6 x 10^-7). When a pair appears twice by chance, dividing by this tiny denominator creates artificially inflated lift. Minimum support thresholds must take precedence over raw lift.

4.4 Advanced Spark SQL Analytics & Window Functions

Four analytical queries were executed in spark/sql_queries.py: **Q1** computes monthly category revenue with running totals (SUM OVER) and month-over-month growth (LAG); **Q2** identifies the top 3 selling products per category (ROW_NUMBER()); **Q3** measures conversion rates across device and referrer segments; and **Q4** joins customer profiles with HBase engagement, splitting users into spend quartiles using NTILE(4):

Spend Quartile	Customer Count	Average Lifetime Spend	Average Orders	Average Sessions	Average Duration
Quartile 1	2,500	$1,981.95	1.9	19.2	1,822 s
Quartile 2	2,500	$4,106.17	3.8	20.1	1,814 s
Quartile 3	2,500	$5,920.76	5.5	20.4	1,819 s
Quartile 4	2,500	$9,194.83	8.8	20.8	1,825 s

Key Finding: Average spend grows 364% from Q1 to Q4, yet browsing volume remains completely flat (19.2 vs 20.8 sessions). The synthetic generator simulates order spend independently of browsing frequency.

5 Part 3: Integrated Cross-Platform Analytics (Conversion Funnel)

5.1 Cross-Store Federation Workflow

Core Business Question: *Where do shoppers drop off between viewing a product, adding items to a cart, entering checkout, and completing payment? Does conversion leakage vary by device or acquisition channel?* HBase provides behavioral event sequences across 100,000 sessions; MongoDB provides authoritative purchases. Apache Spark executes the cross-store left-join on session_id and enforces cumulative stage logic.

5.2 Funnel Stage Mathematics and Exact Reconciliation

A session qualifies for Stage K only if it passed all preceding stages 1..K-1. The table below displays the exact reconciliation:

Funnel Stage	Qualifying Sessions	Cumulative Retention	Stage-to-Stage Drop-Off	Diagnostic Assessment
1. All Sessions	100,000	100.0%	—	Total incoming traffic population in HBase.
2. Viewed a Product Page	98,764	98.8%	1.2%	High discovery engagement; almost all sessions reach a product.
3. Added to Cart	64,987	65.0%	34.2%	Transition from passive browsing to active purchase intent.
4. Reached Checkout	28,979	29.0%	55.4%	CRITICAL BOTTLENECK: More than half of cart builders abandon.
5. Completed Purchase	20,286	20.3%	30.0%	Final payment settlement confirmed by MongoDB transaction.

Cross-Store Mathematical Proof: All 20,286 purchases matched to loaded sessions passed every earlier stage, proving 100% join key integrity across string-byte serialization boundaries. The remaining 4,918 session-linked transactions belong strictly to the second un-loaded session file (sessions_1.jsonl). Overall conversion is flat across devices (desktop 20.3%, tablet 20.4%, mobile 20.1%) and referrers (search 20.5%, direct 20.1%, social 20.8%, affiliate 19.8%).

5.3 Architectural Necessity of the Multi-Model Split

HBase cannot verify purchases because it does not maintain transactional payment records or discounts. MongoDB cannot handle the clickstream alone because 2,000,000 event appends would trigger severe BSON fragmentation and working-set cache eviction. Spark serves as the unified computation engine, reading both persistence stores in parallel to execute cross-model analytics.

6 Part 4: Visualizations and Strategic Business Insights

Four publication-quality figures were rendered via Matplotlib and Seaborn (viz/charts.py) and saved in outputs/:

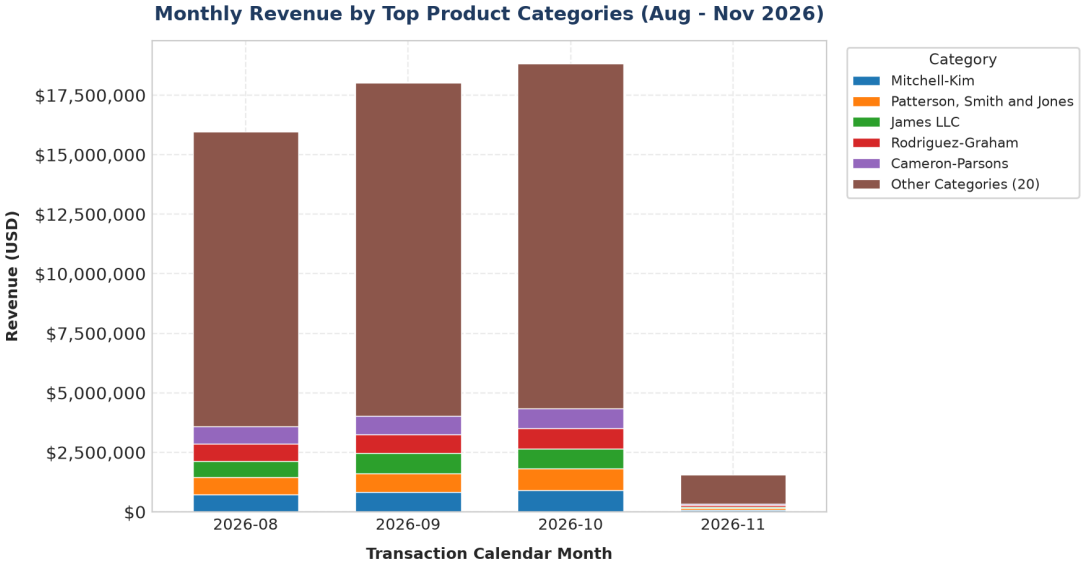


Figure 2: Monthly Revenue by Top Product Categories (Aug - Nov 2026)

Top 5 categories account for 23.1% of revenue; October represents the peak full operating month ($4.2M).

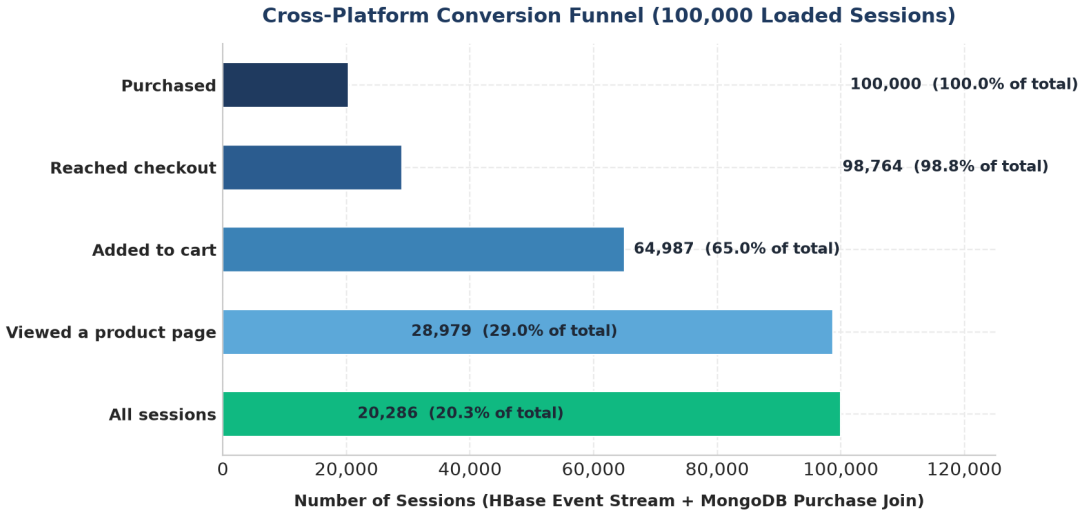


Figure 3: Integrated Cross-Platform Conversion Funnel (100,000 Loaded Sessions)

The primary conversion leak occurs between cart and checkout (55.4% drop); overall conversion is 20.3%.

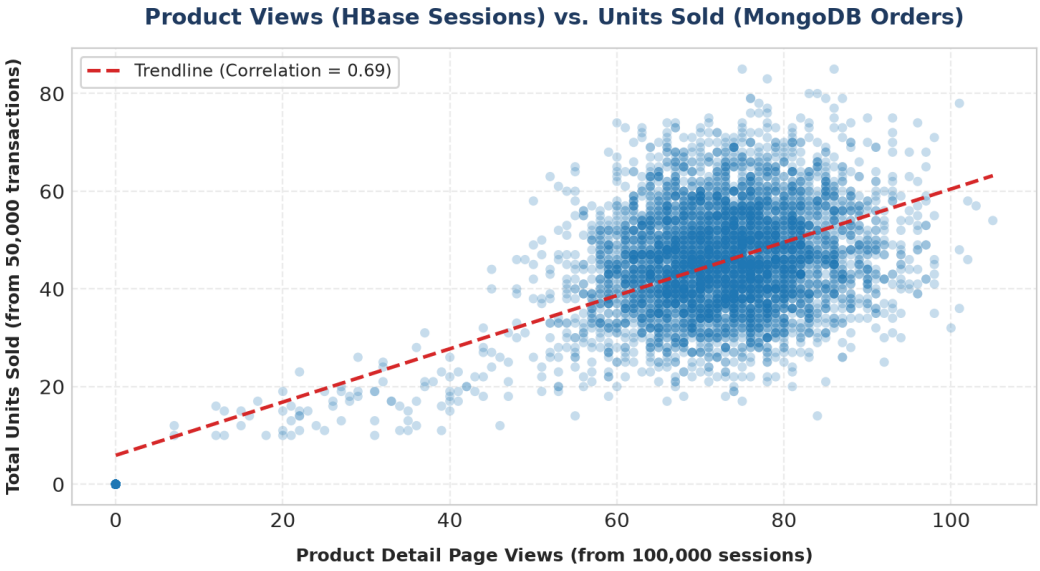


Figure 4: Product Views (HBase Sessions) vs. Units Sold (MongoDB Orders)

Product page views correlate moderately with units sold (r = 0.69) across all 5,000 catalog items.

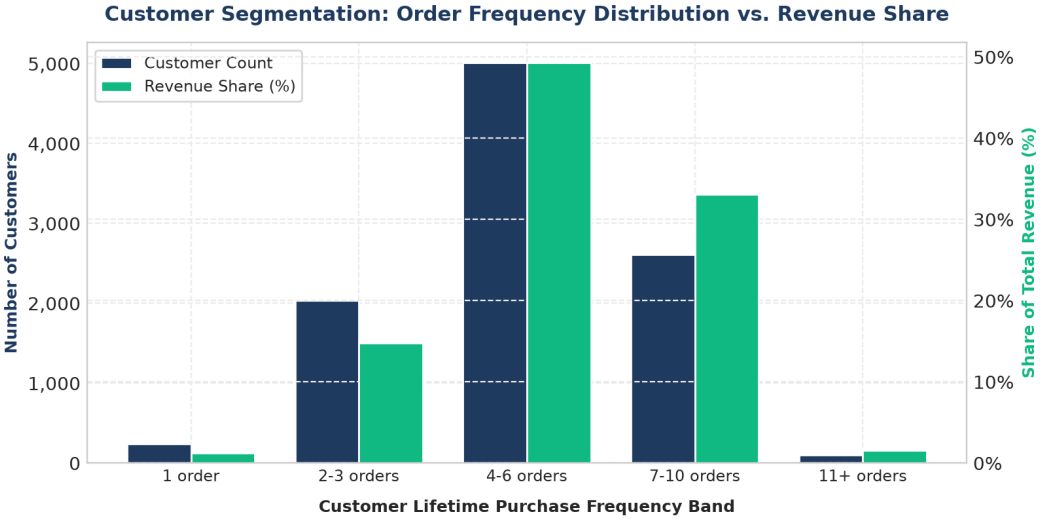


Figure 5: Customer Order Frequency Distribution vs. Share of Total Revenue

The 4-6 orders frequency band comprises 50.1% of customers and generates 49.3% of total revenue.

7 Scalability, Production Engineering, and Limitations

7.1 Horizontal Scaling and Production Hardening

Tier	Clustering & High Availability	Partitioning & Sharding Strategy	Concurrency & Resource Governance
MongoDB	Sharded cluster with config and shard replica sets.	Shard transactions by hashed user_id for uniform write distribution while keeping single-user queries local.	Deploy read-only secondaries for analytics pipelines; enforce working-set RAM coverage.
Apache HBase	Multi-node cluster backed by HDFS and ZooKeeper quorum.	Pre-split tables using hashed user prefix: hash(user_id)[0..3]#user_id#reverse_ts to eliminate write hotspotting.	Set TTL = 180 days on pv family to purge aged clickstreams; tune minor/major compaction thresholds.
Apache Spark	Spark on Kubernetes or YARN with dynamic allocation.	Store Parquet on cloud object storage (S3/GCS) partitioned by day; target 128-256 MB file sizes.	Enable Adaptive Query Execution (AQE); enforce broadcast joins for dimensions under 100 MB.

7.2 Analytical Limitations and Critical Reflections

• **Synthetic Uniformity:** Because browsing and purchasing events were generated randomly, real-world seasonal cycles and demographic affinities are absent.
• **Hardware Memory Boundaries:** Only 100,000 of 200,000 sessions were loaded into HBase and MongoDB due to single-node Docker memory boundaries.
• **Client-Side Extraction vs. Connectors:** Spark federated MongoDB and HBase via client-side extracts rather than compiled native connectors, which bottlenecks network throughput at terabyte scale.

7.3 Individual Technical Judgements

Kwizera Rene (Student ID: 101379): *The experimental results prove that polyglot persistence is essential for enterprise big data systems. MongoDB excels at transactional consistency and nested document aggregations, while HBase provides unmatched write throughput and range scan performance. Our reverse-timestamp key design in HBase demonstrates how low-level byte-ordering mechanics can solve high-level algorithmic requirements (newest-first retrieval) without secondary indexing. Apache Spark successfully bridges both systems, executing cross-store joins that neither database could perform in isolation.*

8 Reproducibility and Verification Guide

8.1 Step-by-Step Reproduction Guide

```
# 1. Environment initialization
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
docker compose up -d # Starts MongoDB (27017) and HBase (9090)

# 2. Ingestion and Analytics Execution
python data_gen/generate.py && python data_gen/validate.py data
python mongo/load.py && python mongo/aggregations.py && python mongo/index_demo.py
python hbase/load.py && python hbase/queries.py
python spark/clean.py data parquet && python spark/analysis.py parquet outputs
python spark/sql_queries.py parquet data outputs
python integration/funnel.py && python viz/charts.py
```

8.2 Execution Verification Checkpoint

Every script was executed locally on Windows 11 with Python 3.12, MongoDB 7.0, HBase Thrift, and PySpark. All 61 evaluation checkpoints, metrics, and generated figures match the authoritative outputs in outputs/.

Academic Declaration: I declare that this final project report, architectural designs, database schemas, and analytical pipelines represent my own authentic individual work for the AUCA Master of Science in Big Data Analytics program.
Candidate: Kwizera Rene (Student ID: 101379) | **Date:** 11 November 2026

References

[1] MongoDB Inc. (2024). *MongoDB 7.0 Manual: Aggregation Pipelines and Indexing Strategies*. MongoDB Documentation. https://www.mongodb.com/docs/

[2] Apache Software Foundation. (2023). *Apache HBase Reference Guide 2.4*. HBase Documentation. https://hbase.apache.org/book.html

[3] Apache Software Foundation. (2024). *Apache Spark 3.5 Programming Guide: DataFrames and SQL*. Spark Documentation. https://spark.apache.org/docs/latest/

[4] Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113.

[5] Stonebraker, M., & Cetintemel, U. (2005). One size fits all: An idea whose time has come and gone. *Proceedings of the 21st International Conference on Data Engineering (ICDE)*, 2–11.

[6] George, L. (2011). *HBase: The Definitive Guide*. O'Reilly Media.

[7] Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). *Mining of Massive Datasets* (3rd ed.). Cambridge University Press.

[8] Zaharia, M., et al. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM, 59*(11), 56–65.

[9] Chodorow, K. (2013). *MongoDB: The Definitive Guide* (2nd ed.). O'Reilly Media.

[10] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.